In Phase 2, your pod will turn the problem analysis from Phase 1 into a common project specification. Your team will then design its own approach to realizing that specification and plan the remainder of the project.

By the end of this phase, every team in your pod should agree on **what system is being built and what it must do**, while remaining free to make different engineering decisions about **how to build it**.

This common specification is important. In later phases, teams will independently implement the system and test one another's implementations. The specification must therefore be precise enough that another team can develop tests against it without knowing how your team implemented the system. Concretely, part of your team's grade is calculated based on how well your team's dynamic tests apply to your podmates' solutions, and vice versa.

Within your Pod

Step 1 — Select the Common Problem

As a pod, review the candidate problems and system concepts developed during Phase 1.

Select a common problem that:

- Is meaningfully connected to your assigned regulation
- Has identifiable users or stakeholders
- Can be addressed by a software system
- Is substantial enough to require meaningful engineering decisions
- Can plausibly be implemented during the semester
- Allows multiple teams to develop independent solutions
- Produces behavior or outputs that can be evaluated independently

You may combine or revise ideas from Phase 1. You are not required to select one team's proposal unchanged.

Document why the pod selected this problem and what alternatives were considered.

Step 2 — Establish Scope

Define the boundary of the common system.

Identify:

- What the system is responsible for
- What is explicitly outside its scope
- Who or what interacts with the system
- Important inputs and outputs
- External systems or services with which it interacts
- Important assumptions about its operating environment
- Regulatory obligations the system is intended to address

PLEASE PLEASE PLEASE be aggressive about scope. A smaller system that can be engineered, validated, and shipped is preferable to an ambitious system that exists primarily as unfinished functionality.

Your articulation of scope should include a cost estimate, and that estimate should account for the GenAI capabilities available to your teams.

Step 3 — Develop the Common Specification

As a pod, develop a specification that every team's implementation must satisfy.

The specification should capture consequential requirements, including as appropriate:

- Required system behavior
- Inputs and outputs
- Interfaces or protocols needed for interoperability and testing
- Important data and state
- Error and failure behavior
- Security requirements
- Regulatory requirements
- Quality attributes that materially affect whether the system is acceptable
- Constraints necessary for cross-team testing

Represent the specification using appropriate **models**. Different aspects of the system may require different representations: requirements, state machines, schemas, interface definitions, workflows, threat models, examples, invariants, or other forms.

The models are part of the specification. They should make important engineering knowledge explicit enough that teams and software agents can reason from it.

Step 4 — Define Acceptance Criteria

Determine how you will know whether an implementation satisfies the common specification.

For each consequential requirement, identify evidence that could establish whether it has been satisfied.

Where practical, define criteria that can be evaluated automatically.

Consider:

- Functional tests
- Property or invariant checks
- Static analyses
- Security tests
- Performance measurements
- Conformance tests
- Inspection of generated artifacts
- Regulatory compliance checks

Pay particular attention to **observable behavior**. Later in the semester, your podmates must be able to test your implementation without depending on knowledge of its internal design.

Your acceptance criteria will evolve as you learn more, but Phase 2 should establish a credible initial basis for validation.

Step 5 — Establish a Cross-Team Testing Interface

Your pod must define a common **web-service interface** that every team implementing the specification will provide.

The purpose of this interface is to permit automated cross-team testing. An independently written test must be able to invoke any team's deployed implementation over the network, provide inputs, and observe outputs without knowing anything about that team's internal architecture.

Specify, as appropriate:

- the HTTP endpoints that must be available;
- request methods and input formats;
- response formats and status codes;
- any required data schemas;

- initialization or reset operations needed for repeatable testing;
- authentication or other access requirements, if any; and
- any externally observable behavior needed to determine whether the implementation satisfies the specification.

The interface should expose enough of the system's consequential behavior to support meaningful automated testing. It does **not** need to expose every feature of the user interface or every internal operation.

All teams must deploy a network-accessible implementation of this interface beginning with Phase 3. A desktop application, mobile application, command-line tool, or other client may still be part of the team's system (and especially for prototyping), but it does not substitute for the required web service used for automated evaluation.

Do not standardize internal architecture or implementation choices merely to make testing easier. Standardize the externally observable boundary necessary for independent implementations to be evaluated against the common specification.

You may wish to use an interface-specification language such as **OpenAPI** to define the required service boundary precisely and make it easier for teams to generate clients, validate requests and responses, and construct automated tests.

Step 6 — Define Phases 3–5

Your pod will define its own milestones for Phases 3, 4, and 5.

Each milestone should represent a meaningful increment of engineering progress rather than simply an amount of implementation work.

For each phase, specify:

- What capability or engineering objective will be completed
- What artifacts will be produced or updated
- What uncertainty or risk the work addresses
- How completion will be demonstrated
- What automated validation will be added

Sequence the milestones deliberately. Use early phases to resolve important uncertainty and establish foundations on which later work depends.

The instructors will review your proposed milestones and may require changes to scope or sequencing.

Your pod may adjust this plan as the semester advances.

Within your team

Step 7 — Design Your Team's System

Once the pod specification is established, each team will independently design its implementation.

Develop an initial design addressing:

- Major components and responsibilities
- Important data and control flows
- External dependencies
- Persistence and state
- Interfaces
- Security boundaries
- Deployment and operation
- Observability

- Validation strategy

Document important design decisions and their rationale, particularly where meaningful alternatives exist.

Your design is expected to change as you learn. The purpose of the Phase 2 design is to establish a reasoned starting point, not to predict every implementation detail.

Step 8 — Decide What to Build, Buy, and Reuse

Identify major capabilities your system requires and determine how you expect to obtain them.

For each significant dependency or capability, consider whether you should:

- Build it yourselves
- Use an existing library or framework
- Integrate an open-source system
- Purchase or use a hosted service
- Delegate suitable work to a Generative AI service

Consider cost, licensing, security, reliability, maintainability, integration effort, and the consequences of depending on the component.

Reuse is encouraged when it allows your engineering effort to focus on the distinctive problems of your project.

Step 9 — Identify Major Risks

Identify the uncertainties most likely to prevent the project from succeeding.

These might include:

- Uncertain technical feasibility
- Ambiguous regulatory interpretation
- Difficult external integrations
- Dependence on unreliable AI behavior
- Security or privacy concerns
- Performance or scalability
- Lack of suitable test data
- Difficult-to-automate validation
- Dependencies outside your control

For each major risk, identify how you intend to reduce the uncertainty. High-risk assumptions should generally be investigated earlier rather than later.